

YAPILAR, BİRLİKLER VE NİTELİKLER

Yapı Tanımlaması

Yapısal programlamaya adını veren **yapı** (**struct**), **türetilmiş** (**derived**) veya **kullanıcı tanımlı** (**user defined**) bir veri tipidir. Farklı tiplerdeki **değer tipi** (**value type**) elemanlarının bir arada gruplandırılan özel bir veri tipi tanımlamak için **struct** anahtar kelimesini kullanırız.

Yapı bir veya birden fazla temel değer veri tipinin bir araya gelmesiyle oluşturulan yeni veri tipleridir. Yapılar farklı dipte elemanların bir araya gelmesiyle oluşabilir. Sınıflara benzer şekilde tanımlanır;

```
erişim-değiştiricisi struct yapı-kimliği {  
    veri-tipi yapı-elemanı-kimliği1; //alan (field)  
    veri-tipi yapı-elemanı-kimliği1; //alan (field)  
    ...  
}
```

Örnek olarak Öğrenci; adı, soyadı, numarası, yaşı, cinsiyeti gibi farklı öğeler ile bir **ogrenci** yapısı tanımlanabilir;

```
public struct Öğrenci {  
    public uint yas;  
    public char cinsiyet;  
    public float kilo;  
    public uint boy;  
}
```

Benzer şekilde koordinat bilgisi içeren yapı aşağıdaki şekilde tanımlanabilir;

```
public struct Koordinat {  
    public double X;  
    public double Y;  
}
```

Yapı Değişkenleri

Tanımlanan yapı kullanılarak değişkenler veya **örnekler** (**instance**) aşağıdaki gibi tanımlanabilir;

```
yapı-kimliği değişken-kimliği;  
//yada  
yapı-kimliği değişken-kimliği=new yapı-kimliği();
```

Tanımlanan yapı değişkenleri üzerinden her bir alana **nokta işlecisi** (**dot operator**) ile erişilir. Bu işleç de parantez işlecisi ile aynı önceliktedir. Ayrıca Atama (=) işlecisi bir **yapıyı** (**struct**) doğrudan kopyalamak için kullanılabilir. Ayrıca, bir yapının üyesinin değerini başka birine atamak için atama işlecini de kullanabiliriz. Yukarıda örneği verilen yapılardan birçok değişken kimliklendirilebilir;

```
Öğrenci ilhan;  
ilhan.yas = 20;  
ilhan.cinsiyet = 'E';  
ilhan.kilo = 80;  
ilhan.boy = 180;  
  
Öğrenci mehmet=new Öğrenci();  
mehmet.yas = 25;  
mehmet.cinsiyet = 'E';  
mehmet.kilo = 75;  
mehmet.boy = 175;  
  
Koordinat nokta1;  
nokta1.X = 10.5;  
nokta1.Y = 20.5;
```

```
Koordinat nokta2= new Koordinat();
nokta2.X = 30.5;
nokta2.Y = 40.5;

nokta2 = nokta1;
```

Yapılarda Yapıcılar

Sınıflardan ön tanımlı nesneler imal edilebildiği gibi yapılardan da ön tanımlı **örnekler** (*instance*) oluşturulabilir. Bunun için **yapıcılar** (*constructor*) kullanılır. Yapı alanları parametrelili bir yapıcı aracılığıyla veya yapı imal edildikten sonra ayrı ayarlanabilir.

```
// See https://aka.ms/new-console-template for more information
Koordinat nokta1;
nokta1.X = 10.5;
nokta1.Y = 20.5;
Console.WriteLine($"Nokta 1: ({nokta1.X}, {nokta1.Y})");
Koordinat nokta2 = new Koordinat();
nokta2.X = 30.5;
nokta2.Y = 40.5;
Console.WriteLine($"Nokta 2: ({nokta2.X}, {nokta2.Y})");
nokta2 = nokta1;
Console.WriteLine("Nokta 2: {0}, {1}", nokta2.X, nokta2.Y);

public struct Koordinat
{
    public double X;
    public double Y;
    public Koordinat()
    {
        X = 0;
        Y = 0;
    }
    public Koordinat(double x, double y)
    {
        X = x;
        Y = y;
    }
}
```

Mahrem (*private*) üyeler yalnızca yapıcı içinde ilk değer verilebilir yani **başlatılabilir** (*initialize*).

Diğer değer tipleri gibi yapılar da **null** olamaz. Ancak değişken olarak **null** olabilir olarak tanımlanabilir;

```
Koordinat nokta1 = null; //HATA!: Değer tiplere null atanamaz!
Koordinat? v2 = null; //OK
Nullable<Koordinat> v3 = null // OK
```

Yapılarda Ara Yüzü Gerçekleştirme

Tanımlanan her **yapı** (*struct*), **System.ValueType** tipinden dolayı olarak miras alan **kısır** (*sealed*) bir veri tipidir. Yapılar başka herhangi bir tipten miras alamaz, ancak **ara yüzleri gerçekleştirebilirler** (*interface realization*);

```
// See https://aka.ms/new-console-template for more information
Dikdörtgen dikdörtgen = new Dikdörtgen(5, 10);
Console.WriteLine($"Dikdörtgenin alanı: {dikdörtgen.Alan()}");
Daire daire = new Daire(7);
Console.WriteLine($"Dairenin alanı: {daire.Alan()}");
public interface IŞekil // Bir arayüz
{
    double Alan();
}
```

```

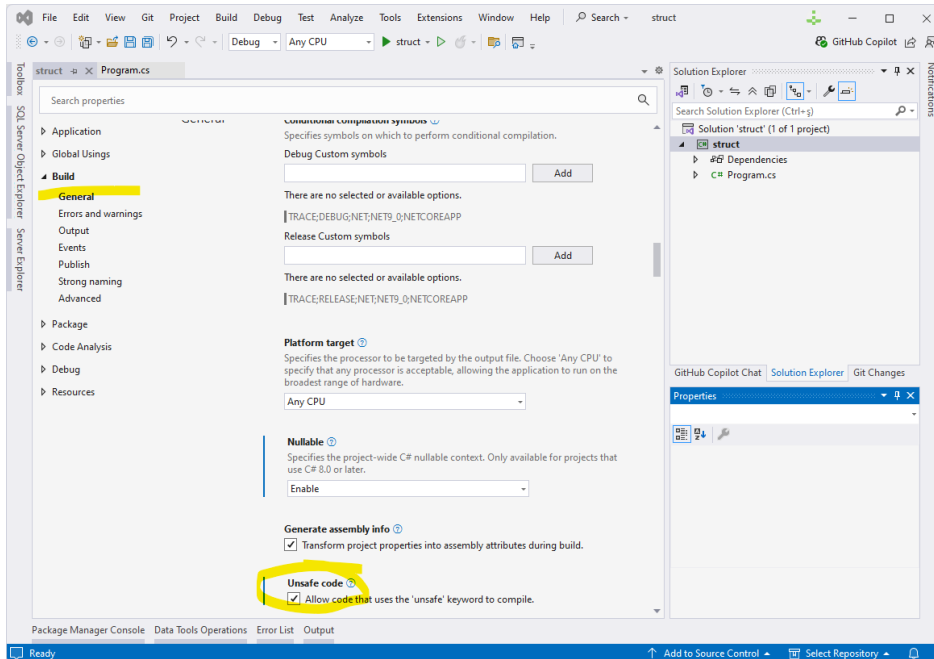
}
public struct Daire : IŞekil // Daire yapısı IŞekil arayüzünü gerçekleştiriyor
{
    public double yarıcap;
    public Daire(double yarıcap)
    {
        this.yarıcap = yarıcap;
    }
    public double Alan()
    {
        return Math.PI * yarıcap * yarıcap;
    }
}
public struct Dikdörtgen : IŞekil // Dikdörtgen yapısı IŞekil arayüzünü uyguluyor
{
    public double uzunKenar;
    public double kısaKenar;
    public Dikdörtgen(double uzunKenar, double kısaKenar)
    {
        this.uzunKenar = uzunKenar;
        this.kısaKenar = kısaKenar;
    }
    public readonly double Alan()
    {
        return uzunKenar * kısaKenar;
    }
}
}

```

Sınıflarda olduğu gibi bir yapıya üye yöntem tanımlanabilir. Eğer üye yöntem bir yapı alanını değiştirmeyecek ise yukarıdaki gibi **readonly** tanımlanabilir.

Boyutu Sabit Dizi Elemanına Sahip Yapılar

C# dilinin önceki sürümlerinde C++ dilindeki gibi sabit boyutlu bir yapı bildirmek zordur. Ancak C# dilinin gelişmiş sürümlerinde sabit boyutlu diziler mümkündür.



Şekil 33. Güvenli Olmayan Kod Derlenmesi

Artık güvenli olmayan kod blokları içerisinde dizileri sabit boyutlu olarak bildirmek mümkündür; Bunun için projenin Visual Studio içerisinde **unsafe** olarak derleneceğinin bildirilmesi gerekir. Visual Studio içerisinde ise proje özelliklerinde güvenli olmayan kodun derlenmesi için değişiklikler yapılmalıdır. Bir

projenin güvenli olmayan şekilde derlenebilmesi için komut satırında `/unsafe` argümanı derleyiciye verilmelidir;

```
C:\Users\ILHANQZKAN>csc Program.cs /unsafe
```

Aşağıda sabit boyutlu bir diziye sahip bir yapı örneği verilmiştir;

```
// See https://aka.ms/new-console-template for more information
unsafe
{
    StructWithFixedArray s = new StructWithFixedArray();
    s.aNumber = 5;
    s.anArray[0] = 1.0f;
    s.anArray[1] = 2.0f;
    s.anArray[2] = 3.0f;
    s.anArray[3] = 4.0f;
}
public unsafe struct StructWithFixedArray
{
    public int aNumber;
    public fixed float anArray[5];
}
```

Birlikler

Birlikler (**union**), C dili gibi çeşitli dillerde, aynı bellek bölgesinde örtüşebilen birkaç farklı türü içermek için kullanılır. Başka bir deyişle, farklı uzunluklara ve türlere sahip olsalar bile, hepsi aynı bellek adresinde başlayan farklı alanlar içerebilirler. Bunun hem bellek tasarrufu hem de otomatik dönüştürme yapma avantajı vardır. Örnek olarak bir IP adresini düşünün. Dahili olarak, bir IP adresi bir tam sayı olarak gösterilir, ancak bazen **Bayt1.Bayt2.Bayt3.Bayt4** şeklinde farklı bayt bileşenine erişmek isteriz. Bu, **Int32** veya **long** gibi ilkel değer tipler veya kendiniz tanımladığınız diğer yapılar (**struct**) olsun, herhangi bir değer tipi için işe yarar;

```
// See https://aka.ms/new-console-template for more information
using System.Runtime.InteropServices;
var ip = new IPAdresi(new Random().Next());
Console.WriteLine($"{ip} = {ip.ipAdresi}"); //33.62.229.21 = 367345185
ip.Bayt1 = 100;
Console.WriteLine($"{ip} = {ip.ipAdresi}"); //100.62.229.21 = 367345252

[StructLayout(LayoutKind.Explicit)]
struct IPAdresi
{
    // "FieldOffset" ipAdresi tamsayısının adresidir.
    // sizeof(int) 4, sizeof(byte) = 1
    [FieldOffset(0)] public int ipAdresi;
    [FieldOffset(0)] public byte Bayt1; // Yapılın 1. byte'ı offset 0
    [FieldOffset(1)] public byte Bayt2; // Yapılın 2. byte'ı offset 1
    [FieldOffset(2)] public byte Bayt3; // Yapılın 2. byte'ı offset 2
    [FieldOffset(3)] public byte Bayt4; // Yapılın 2. byte'ı offset 3
    public IPAdresi(int address) : this()
    {
        ipAdresi = address;
    }
    public override string ToString() => $"{Bayt1}.{Bayt2}.{Bayt3}.{Bayt4}";
}
```

Bir **yapıyı** (**struct**), C dilindeki **birlik** (**union**) gibi kullanmak için yapının üzerine `[StructLayout(LayoutKind.Explicit)]` **niteliğini** (**attribute**) eklemek gerekir. Nitelikler aslında **cephe yönelimli programlamanın** (**aspect oriented programming-AOP**) C# diline uygulanmış halidir.

Nitelikler

Nitelikler (**attribute**), C# dilinde programcıların **çalıştırma anında** (**run time**) erişip kullanabilmeleri için, **bileşenlere** (sınıf, alan, yöntem) **tanıtıcı** (**declarative**) ifadeler eklenmesini sağlarlar. Aslında nitelikler, programcıya yardımcı olan ek özelliklerdir.

Nitelik kavramını bir örnekle açıklayalım; Bazı durumlarda sınıflarda tanımlanan alanın adı, veri tabanında farklı, bir başka bileşen için farklı olabilmektedir. Böyle durumlarda bunun veri tabanındaki adını bir nitelik olarak tanımlayıp, çalıştırma anında programcı bu niteliği kullanarak, alana ilişkin değeri veri tabanına kaydedebilir veya veri tabanından okuyabilir.

Bu kavram aynı zamanda farklı zamanlarda ve platformlarda geliştirilmiş bileşenleri birlikte kullanmak için programcıya oldukça yardımcı olur.

Tüm nitelikler **System.Attribute** sınıfından devralınmalıdır. Nitelik kullanımınızı **System.AttributeUsage** kullanarak özelleştirebilirsiniz.

```
// See https://aka.ms/new-console-template for more information
using System.Reflection;
//Sınıfın Niteliklerini Alma:
foreach (Attribute? attribute in typeof(NitelikKullananSınıf).GetCustomAttributes())
{
    Console.WriteLine(attribute.GetType());
}
//Nitelik Detayını Alma:
var attribute2 =
typeof(NitelikKullananSınıf).GetCustomAttributes().OfType<AçıklamaNiteliği>().Single();
Console.WriteLine(attribute2.remark);
var attribute3 = typeof(NitelikKullananSınıf).GetCustomAttribute<AçıklamaNiteliği>();
Console.WriteLine(attribute3.remark);

[AttributeUsage(AttributeTargets.All)]
class AçıklamaNiteliği : Attribute
{
    string pri_remark;
    public AçıklamaNiteliği(string comment)
    {
        pri_remark = comment;
    }
    public string remark
    {
        get
        {
            return pri_remark;
        }
    }
}

[AçıklamaNiteliği("Bu sınıf özel nitelik kullanıyor..")]
class NitelikKullananSınıf
{
    // ...
}
```

Bir yöntemi çağırdığımızda iz kaydı alınacak bir nitelik aşağıdaki şekilde yazılabilir;

```
// See https://aka.ms/new-console-template for more information
var service = new MyService();
service.İzKaydıAlınmayacakYöntem(); // Bu yöntemde iz kaydı alınmaz!
service.İzKaydıAlınacakYöntem("Çay Demle"); /* Bu şekilde Çağrılırsa bu yöntemde
iz kaydı alınmaz! */
// İz kaydı alınacak yöntemi çağırmak için aşağıdaki gibi bir yöntem kullanılabilir:
İzKaydıYardımcısı.İzkaydıOlanYöntemiÇağır(service, "İzKaydıAlınacakYöntem",
"Çağrılan Yönteme Bu Parametreyi Gönder");
```

```
[AttributeUsage(AttributeTargets.Method)]
public class İzKaydıNiteliği : Attribute
{
}
public class MyService
{
    [İzKaydıNiteliği]
    public void İzKaydıAlınacakYöntem(string yapılanış)
    {
        Console.WriteLine($"Yöntem, \"{yapılanış}\" işini yapıyor...");
    }
    public void İzKaydıAlınmayacakYöntem()
    {
        Console.WriteLine("Bu yöntemde yapılan işlerin iz kaydı alınmaz!");
    }
}
public static class İzKaydıYardımcısı
{
    public static void İzKaydıOlanYöntemiÇağır(object target,
                                                string methodName,
                                                params object[] parameters)
    {
        var method = target.GetType().GetMethod(methodName);
        if (method == null) throw new ArgumentException("Böyle bir yöntem yok!");
        if (method.GetCustomAttributes(typeof(İzKaydıNiteliği), true).Any())
        {
            Console.WriteLine($"[LOG]:Çağrılacak Yöntemin;");
            Console.WriteLine($"Adı: {method.Name}");
            Console.WriteLine($"Pacapetreleri:{string.Join(", ", parameters)}");
        }
        method.Invoke(target, parameters); //Yöntemi Çağır!
    }
}
```

Bu manuel yaklaşım sınırlıdır ve büyük projeler için ideal değildir. ASP.NET Core için **ActionFilterAttribute** veya **ILogger** kullanılır. Gerçek Cephe Yönelimli Programlama (AOP) tabanlı günlük kaydı için **PostSharp/Fody** kullanılır. Gerçek **günlük (log)** çıktısı için **Serilog**, **NLog** veya **log4net** bileşenlerinden biri kullanılır.

Bu tür bileşenleri Visual Studio üzerinde projemize dahil etmek için *Tools → NuGet Package Manager* seçilerek açılan konsolda komut vererek bileşenleri projemize yüklememiz gerekir. Aşağıda **Foddy** bileşenini projemize dahil etmek için gerekli **paket yöneticisi (package manager)** komutları verilmiştir;

```
Install-Package Fody
Install-Package MethodDecorator.Fody
```

Eğer .NET Core CLI ile proje oluşturulacak ise aşağıdaki adımlar boş bir klasörde yazılarak takip edilir;

```
D:\CSharp\Samples\foddy>dotnet new console
D:\CSharp\Samples\foddy>dotnet add package Fody
D:\CSharp\Samples\foddy>dotnet add package MethodDecorator.Fody
```

Bu paket yüklendikten sonra yardımcı sınıfa gerek kalmadan nitelikler doğrudan iz kaydı alabilir olacaktır. Örnek **Program.cs** dosyası aşağıda verilmiştir;

```
// See https://aka.ms/new-console-template for more information
using MethodDecorator.Fody.Interfaces;
using System.Reflection;

var service = new MyService();
service.Process("Hello World");
try
{
}
```

```

        service.Crash();
    }
    catch (Exception ex)
    {
        Console.WriteLine($"Caught exception: {ex.Message}");
    }
}
/* Program Çıktısı:
[LOG] Entering MyService.Process with args: Hello World
Processing: Hello World
[LOG] Exiting MyService.Process
[LOG] Entering MyService.Crash with args:
[LOG] Exception in MyService.Crash: Something went wrong!
Caught exception: Something went wrong!
*/

[AttributeUsage(AttributeTargets.Method | AttributeTargets.Constructor |
AttributeTargets.Assembly | AttributeTargets.Module)]
public class LoggerAttribute : Attribute, IMethodDecorator
{
    private MethodBase _method;
    private object[] _args;
    public void Init(object instance, MethodBase method, object[] args)
    {
        _method = method;
        _args = args;
    }
    public void OnEntry()
    {
        Console.WriteLine($"[LOG] Entering {_method.DeclaringType.Name}.{_method.Name}"+
            $" with args: {string.Join(", ", _args)}");
    }
    public void OnExit()
    {
        Console.WriteLine($"[LOG] Exiting {_method.DeclaringType.Name}.{_method.Name}");
    }
    public void OnException(Exception exception)
    {
        Console.WriteLine($"[LOG] Exception in {_method.DeclaringType.Name}.{_method.Name}: "+
            $" {exception.Message}");
    }
}

public class MyService
{
    [Logger]
    public void Process(string message)
    {
        Console.WriteLine($"Processing: {message}");
    }
    [Logger]
    public void Crash()
    {
        throw new InvalidOperationException("Something went wrong!");
    }
}

```